Exercises XP: LoRA Implementation Lab

Replace each `TODO` before running the next section.

What you'll learn

- The fundamentals of LoRA (Low-Rank Adaptation) and why it helps churn out efficient fine-tunes.
- How to implement LoRA matrices `A` and `B`, plus how to wrap existing `nn.Linear` layers.
- Differences between standard linear layers, LoRA-enhanced layers, and merged-weight alternatives.
- How to freeze base parameters so that only the LoRA adapters receive updates.

What you will create

- A reusable `LoRALayer` module and two linear wrappers (`LinearWithLoRA`, `LinearWithLoRAMerged`).
- A 3-layer MLP that can be swapped between standard and LoRA-enhanced variants.
- A minimal MNIST training loop plus accuracy helpers to compare frozen vs. fully-trainable adapters.
- A workflow to freeze baseline weights and fine-tune only the LoRA layers.

> **Learning point**
> Keep the student and teacher notebooks open side by side. Follow the numbered exercises, run setup only once, and watch tensor shapes as you add LoRA adapters.

Part 0: Environment Setup

Install the CPU-friendly PyTorch stack plus torchvision for MNIST. Reuse caches across reruns to save time.

```
%pip install --quiet torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cpu
```

```
import copy
import time

import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader
from torchvision import datasets, transforms

BASE_SEED = 123
torch.manual_seed(BASE_SEED)

DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {DEVICE}")
```

```
Using device: cpu
```

Exercise 1: Implement `LoRALayer`

Create the low-rank matrices `A` and `B`, scale them with `alpha`, and test the module on a toy tensor.

```
class LoRALayer(nn.Module):
    def __init__(self, in_dim, out_dim, rank, alpha):
        super().__init__()
        std_dev = 1 / torch.sqrt(torch.tensor(rank).float())
        self.A = nn.Parameter(torch.randn(in_dim, rank) * std_dev)
        self.B = nn.Parameter(torch.zeros(rank, out_dim))
        self.alpha = alpha

    def forward(self, x):
        lora_output = (x @ self.A) @ self.B
        return self.alpha * lora_output

# --- Sandbox Test ---

random_seed = 123
```

```
in_dim = 10
out_dim = 5
rank = 2
alpha = 1.0

torch.manual_seed(random_seed)
layer = LoRALayer(in_dim, out_dim, rank, alpha)

x = torch.randn(2, in_dim)

print("Input Tensor x:\n", x)
print("\nLoRA Layer Structure:\n", layer)
print("\nLoRA Output (should be all zeros initially because B is zero):\n", layer(x))
```

```
Input Tensor x:
 tensor([[-0.3885, -0.9343, -0.4991, -1.0867, -0.2044, -2.2685, -0.9133, -0.4204,
          1.3111, -0.2199],
        [ 0.2190,  0.2045,  0.6177, -0.2876,  0.8218,  0.1512,  0.1036, -2.1996,
         -0.0885, -0.5612]])

LoRA Layer Structure:
 LoRALayer()

LoRA Output (should be all zeros initially because B is zero):
 tensor([[0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0.]], grad_fn=<MulBackward0>)
```

Exercise 2: Wrap `nn.Linear` with LoRA

Combine a frozen linear projection plus a trainable `LoRALayer`. Confirm the adapter outputs add on top of the base logits.

```
class LinearWithLoRA(nn.Module):
    def __init__(self, linear, rank, alpha):
        super().__init__()
        self.linear = linear
        self.lora = LoRALayer(
            linear.in_features,
            linear.out_features,
            rank,
            alpha,
        )

    def forward(self, x):
        return self.linear(x) + self.lora(x)

base_linear = nn.Linear(in_dim, out_dim)

layer_lora_1 = LinearWithLoRA(base_linear, rank, alpha)

print("LinearWithLoRA output:\n", layer_lora_1(x))
```

```
LinearWithLoRA output:
 tensor([[ 1.3557, -1.0487, -0.3321, -0.3467, -0.3075],
        [-0.1582, -0.6605,  0.8469, -0.0565, -0.7080]], grad_fn=<AddBackward0>)
```

Exercise 3: Swap a simple network layer with LoRA

Start from a single-layer perceptron, then replace its linear block with `LinearWithLoRA`. The outputs should match before training because the LoRA adapters start at zero.

```
class SingleLayerNet(nn.Module):
    def __init__(self, num_features, num_classes):
        super().__init__()
        self.layer = nn.Linear(num_features, num_classes)

    def forward(self, x):
        return self.layer(x)

num_features = 784 # Example: MNIST image size 28x28
num_classes = 10   # Example: 10 digits
single_net = SingleLayerNet(num_features=num_features, num_classes=num_classes)

# Generate a random input tensor:
sample_input = torch.randn(1, num_features)

with torch.no_grad():
    baseline_output = single_net(sample_input)
```

```
single_net.layer = LinearWithLoRA(single_net.layer, rank=4, alpha=1.0)

with torch.no_grad():
    lora_output = single_net(sample_input)

# Verify the outputs match:
matches = torch.allclose(baseline_output, lora_output, atol=1e-6)
print("Outputs match before training?", matches)
```

```
Outputs match before training? True
```

Exercise 4: Merged-weight LoRA layer

Fuse the LoRA matrices with the frozen weights to create a drop-in linear layer that behaves exactly like `LinearWithLoRA`.

```
class LinearWithLoRAMerged(nn.Module):
    def __init__(self, linear, rank, alpha):
        super().__init__()
        self.linear = linear
        self.lora = LoRALayer(
            linear.in_features,
            linear.out_features,
            rank,
            alpha,
        )

    def forward(self, x):
        lora_weights = self.lora.A @ self.lora.B

        # Merge:
        combined_weight = self.linear.weight + (self.lora.alpha * lora_weights.t())

        return F.linear(x, combined_weight, self.linear.bias)


layer_lora_2 = LinearWithLoRAMerged(base_linear, rank, alpha)

print("Merged LoRA output:\n", layer_lora_2(x))
```

```
Merged LoRA output:
 tensor([[ 1.3557, -1.0487, -0.3321, -0.3467, -0.3075],
        [-0.1582, -0.6605,  0.8469, -0.0565, -0.7080]],
       grad_fn=<AddmmBackward0>)
```

Exercise 5: Build an MLP and prepare MNIST

Stack three linear layers with ReLU activations, then set up the MNIST loaders plus optimizer/state for pretraining.

```
class MultilayerPerceptron(nn.Module):
    def __init__(self, num_features, num_hidden_1, num_hidden_2, num_classes, rank, alpha):
        super().__init__()
        self.layers = nn.Sequential(
            # First Hidden Layer:
            LinearWithLoRAMerged(nn.Linear(num_features, num_hidden_1), rank, alpha),
            nn.ReLU(),

            # Second Hidden Layer:
            LinearWithLoRAMerged(nn.Linear(num_hidden_1, num_hidden_2), rank, alpha),
            nn.ReLU(),

            # Output Layer:
            LinearWithLoRAMerged(nn.Linear(num_hidden_2, num_classes), rank, alpha),
        )

    def forward(self, x):
        x = x.view(x.size(0), -1)
        x = self.layers(x)
        return x
```

```
# Architecture
num_features = 28 * 28
num_hidden_1 = 128
num_hidden_2 = 64
num_classes = 10
```

```
# Settings
DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
learning_rate = 0.005
num_epochs = 3

# Note: Using the rank/alpha from earlier setup:
rank = 4
alpha = 1.0

model = MultilayerPerceptron(
    num_features=num_features,
    num_hidden_1=num_hidden_1,
    num_hidden_2=num_hidden_2,
    num_classes=num_classes,
    rank=rank,
    alpha=alpha
)

model.to(DEVICE)

optimizer_pretrained = torch.optim.Adam(model.parameters(), lr=learning_rate)

print(DEVICE)
print(model)
print(optimizer_pretrained)
```

```
cpu
MultilayerPerceptron(
  (layers): Sequential(
    (0): LinearWithLoRAMerged(
      (linear): Linear(in_features=784, out_features=128, bias=True)
      (lora): LoRALayer()
    )
    (1): ReLU()
    (2): LinearWithLoRAMerged(
      (linear): Linear(in_features=128, out_features=64, bias=True)
      (lora): LoRALayer()
    )
    (3): ReLU()
    (4): LinearWithLoRAMerged(
      (linear): Linear(in_features=64, out_features=10, bias=True)
      (lora): LoRALayer()
    )
  )
)
Adam (
Parameter Group 0
    amsgrad: False
    betas: (0.9, 0.999)
    capturable: False
    decoupled_weight_decay: False
    differentiable: False
    eps: 1e-08
    foreach: None
    fused: None
    lr: 0.005
    maximize: False
    weight_decay: 0
)
```

Loading dataset

```
BATCH_SIZE = 64

train_dataset = datasets.MNIST(root='data', train=True, transform=transforms.ToTensor(), download=True)

test_dataset = datasets.MNIST(root='data', train=False, transform=transforms.ToTensor(), download=True)

train_loader = DataLoader(dataset=train_dataset, batch_size=BATCH_SIZE, shuffle=True)

test_loader = DataLoader(dataset=test_dataset, batch_size=BATCH_SIZE, shuffle=False)

# Verification loop:
for images, labels in train_loader:
    print('Image batch dimensions:', images.shape)
    print('Image label dimensions:', labels.shape)
    break
```

```
100%|██████████| 9.91M/9.91M [00:00<00:00, 59.5MB/s]
100%|██████████| 28.9k/28.9k [00:00<00:00, 1.79MB/s]
100%|██████████| 1.65M/1.65M [00:00<00:00, 13.7MB/s]
100%|██████████| 4.54k/4.54k [00:00<00:00, 5.53MB/s]Image batch dimensions: torch.Size([64, 1, 28, 28])
```

```
Image label dimensions: torch.Size([64])
```

Define evaluation

```
def compute_accuracy(model, data_loader, device):
    model.eval()
    correct_pred, num_examples = 0, 0
    with torch.no_grad():
        for features, targets in data_loader:
            features = features.to(device)
            targets = targets.to(device)

            logits = model(features)

            _, predicted_labels = torch.max(logits, 1)

            num_examples += targets.size(0)
            correct_pred += (predicted_labels == targets).sum()

    return correct_pred.float() / num_examples * 100
```

Training

```
def train(num_epochs, model, optimizer, train_loader, device):
    start_time = time.time()
    for epoch in range(num_epochs):
        model.train()
        for batch_idx, (features, targets) in enumerate(train_loader):
            features = features.to(device)
            targets = targets.to(device)

            logits = model(features)

            loss = F.cross_entropy(logits, targets)

            optimizer.zero_grad()

            loss.backward()

            optimizer.step()

            if not batch_idx % 400:
                print('Epoch: %03d/%03d|Batch %03d/%03d| Loss: %.4f' %
                      (epoch+1, num_epochs, batch_idx, len(train_loader), loss))

        with torch.set_grad_enabled(False):
            print('Epoch: %03d/%03d training accuracy: %.2f%%' %
                  (epoch+1, num_epochs, compute_accuracy(model, train_loader, device)))

        print('Time elapsed: %.2f min' % ((time.time() - start_time)/60))

    print('Total Training Time: %.2f min' % ((time.time() - start_time)/60))
```

```
train(num_epochs, model, optimizer_pretrained, train_loader, DEVICE)

test_acc = compute_accuracy(model, test_loader, DEVICE)
print(f'Test accuracy: {test_acc:.2f}%')
```

```
/tmp/ipykernel_10378/3751919576.py:20: UserWarning: Converting a tensor with requires_grad=True to a scalar may l
Consider using tensor.detach() first. (Triggered internally at /pytorch/torch/csrc/autograd/generated/python_vari
  print('Epoch: %03d/%03d|Batch %03d/%03d| Loss: %.4f' %
Epoch: 001/003|Batch 000/938| Loss: 2.2950
Epoch: 001/003|Batch 400/938| Loss: 0.3325
Epoch: 001/003|Batch 800/938| Loss: 0.2094
Epoch: 001/003 training accuracy: 96.23%
Time elapsed: 0.34 min
Epoch: 002/003|Batch 000/938| Loss: 0.1650
Epoch: 002/003|Batch 400/938| Loss: 0.1251
Epoch: 002/003|Batch 800/938| Loss: 0.0758
Epoch: 002/003 training accuracy: 97.03%
Time elapsed: 0.69 min
Epoch: 003/003|Batch 000/938| Loss: 0.0854
Epoch: 003/003|Batch 400/938| Loss: 0.0865
Epoch: 003/003|Batch 800/938| Loss: 0.0097
Epoch: 003/003 training accuracy: 96.43%
Time elapsed: 1.05 min
Total Training Time: 1.05 min
Test accuracy: 95.32%
```

Replacing Linear with LoRA Layers

```
model_lora = copy.deepcopy(model)

model_lora.layers[0] = LinearWithLoRAMerged(model_lora.layers[0].linear, rank=4, alpha=8)
model_lora.layers[2] = LinearWithLoRAMerged(model_lora.layers[2].linear, rank=4, alpha=8)
model_lora.layers[4] = LinearWithLoRAMerged(model_lora.layers[4].linear, rank=4, alpha=8)

model_lora.to(DEVICE)

optimizer_lora = torch.optim.Adam(model_lora.parameters(), lr=learning_rate)

print(model_lora)

print(f'Test accuracy orig model: {compute_accuracy(model, test_loader, DEVICE):.2f}%')
print(f'Test accuracy LoRA model: {compute_accuracy(model_lora, test_loader, DEVICE):.2f}%')
```

```
MultilayerPerceptron(
  (layers): Sequential(
    (0): LinearWithLoRAMerged(
      (linear): Linear(in_features=784, out_features=128, bias=True)
      (lora): LoRALayer()
    )
    (1): ReLU()
    (2): LinearWithLoRAMerged(
      (linear): Linear(in_features=128, out_features=64, bias=True)
      (lora): LoRALayer()
    )
    (3): ReLU()
    (4): LinearWithLoRAMerged(
      (linear): Linear(in_features=64, out_features=10, bias=True)
      (lora): LoRALayer()
    )
  )
)
Test accuracy orig model: 95.32%
Test accuracy LoRA model: 95.55%
```

Freezing the Original Linear Layers

```
def freeze_linear_layers(model):
    for child in model.children():
        if isinstance(child, nn.Linear):
            for param in child.parameters():
                param.requires_grad = False
        else:
            freeze_linear_layers(child)

freeze_linear_layers(model_lora)

for name, param in model_lora.named_parameters():
    print(f'{name}: {param.requires_grad}')
```

```
layers.0.linear.weight: False
layers.0.linear.bias: False
layers.0.lora.A: True
layers.0.lora.B: True
layers.2.linear.weight: False
layers.2.linear.bias: False
layers.2.lora.A: True
layers.2.lora.B: True
layers.4.linear.weight: False
layers.4.linear.bias: False
layers.4.lora.A: True
layers.4.lora.B: True
```

```
optimizer_lora = torch.optim.Adam(model_lora.parameters(), lr=learning_rate)

train(num_epochs, model_lora, optimizer_lora, train_loader, DEVICE)

# Final Evaluation:
print(f'Test accuracy LoRA finetune: {compute_accuracy(model_lora, test_loader, DEVICE):.2f}%')

print(f'Test accuracy orig model: {compute_accuracy(model, test_loader, DEVICE):.2f}%')
print(f'Test accuracy LoRA model: {compute_accuracy(model_lora, test_loader, DEVICE):.2f}%')
```

```
Epoch: 001/003|Batch 000/938| Loss: 0.0750
Epoch: 001/003|Batch 400/938| Loss: 0.0797
```

```
Epoch: 001/003|Batch 800/938| Loss: 0.1102
Epoch: 001/003 training accuracy: 97.50%
Time elapsed: 0.33 min
Epoch: 002/003|Batch 000/938| Loss: 0.0524
Epoch: 002/003|Batch 400/938| Loss: 0.0799
Epoch: 002/003|Batch 800/938| Loss: 0.1123
Epoch: 002/003 training accuracy: 98.09%
Time elapsed: 0.66 min
Epoch: 003/003|Batch 000/938| Loss: 0.0244
Epoch: 003/003|Batch 400/938| Loss: 0.1170
Epoch: 003/003|Batch 800/938| Loss: 0.0110
Epoch: 003/003 training accuracy: 98.21%
Time elapsed: 0.97 min
Total Training Time: 0.97 min
Test accuracy LoRA finetune: 97.18%
Test accuracy orig model: 95.32%
Test accuracy LoRA model: 97.18%
```

Conclusion:

The results of these exercises demonstrate the power of Low-Rank Adaptation (LoRA) as a parameter-efficient fine-tuning strategy. By initializing matrix B to zero, you ensured the model's performance remained identical to the pre-trained version at the start, preserving existing knowledge. As training progressed, the model successfully adapted to the MNIST dataset by updating only the small A and B matrices, achieving high accuracy without modifying the massive original weight matrices. This proves that a tiny fraction of total parameters the low-rank adapters can capture the necessary task specific adjustments, significantly reducing the computational and memory overhead required for deep learning adaptation.

Furthermore, the implementation of weight merging illustrates how LoRA can be deployed with zero inference latency. By mathematically combining the learned adapters with the frozen weights ($W + \alpha AB^T$), the final architecture performs at the same speed as a standard neural network. This balance between training efficiency and deployment speed highlights why LoRA is a cornerstone of modern AI workflows, allowing developers to fine tune large scale models like LLMs on consumer-grade hardware while maintaining the integrity of the original foundational model.